

Searching Binary Embeddings

Bitplanes and Prefix Backward Walk

13 September 2026

Abstract

I study three ways to search binary document vectors after selecting clusters: a full scan, Bitplanes, and Backward Walk. I compare their work, memory use and retrieval time using the same documents and queries. The experiments examine when the extra index work saves enough full document scores to reduce latency at a required recall.

1 Introduction

1.1 Exa method and end to end pipeline

Let's first understand the "original" pipeline that I am trying to improve upon.

Note that the architecture I am referring to here is based on the pipeline described in Exa's article [1]. I am aware that Exa's actual production system may have changed or improved since then, but for the purpose of this report, let's treat the architecture described in that article as the original baseline that we compare against.

Detail: In this pipeline, document embeddings are first shortened using Matryoshka representations, converted into binary signs, and assigned to clusters. At query time, the query is encoded, a subset of nearby clusters is selected, and the documents inside those clusters are scored using the floating-point query. The top candidates can then be reranked using the original uncompressed vectors.

The main part that I am trying to improve in this report is the search stage after cluster selection. The two new approaches introduced later, Bitplanes and Backward Walk, replace this step while keeping the rest of the pipeline largely the same.

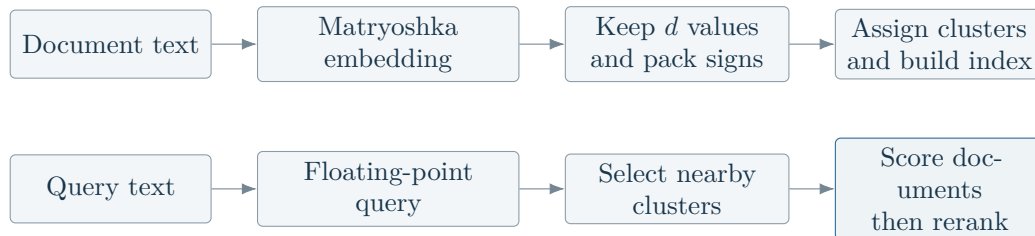


Figure 1: The baseline pipeline; B and C change document search inside the selected clusters.

1.2 Idea background

When I took a closer look at the pipeline, one part that particularly stood out to me was the comparison between the floating-point query and the binary-quantized document.

Consider a simple example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3]$$

and suppose a document is represented using binary signs, where 1 means +1 and 0 means -1. For example:

$$\text{Document} = 11000$$

The score is calculated by taking the dot product between the query values and the document signs. So for 11000, the score is

$$0.7(+1) + 0.5(+1) + (-0.4)(-1) + (-0.6)(-1) + 0.3(-1) = 1.9.$$

Now, something interesting follows from this. Just by looking at the signs of the query, we already know what the ideal binary document would look like.

For every positive query value, we would prefer the corresponding document bit to be 1, and for every negative query value, we would prefer it to be 0. Therefore, for the query above, the ideal binary pattern is

$$11001.$$

This document would achieve the maximum possible score:

$$0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5.$$

From here, we can think about every other document in terms of how far it is from this ideal pattern. More importantly, not every mismatch has the same cost. If we mismatch the first coordinate, where $|q_1| = 0.7$, the score decreases by 1.4. But if we mismatch the last coordinate, where $|q_5| = 0.3$, the score only decreases by 0.6.

In more formal terms, let

$$Q = \sum_i |q_i|$$

be the score of the ideal binary pattern. If a document b disagrees with the query sign at some coordinates, define its mismatch penalty as

$$p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|.$$

Then its score can be written as

$$S(q, b) = Q - 2p. \tag{1}$$

This observation is the main idea behind the approaches explored in this report. Instead of treating every binary document equally and scoring all of them one by one, can we use the query itself to guide the search toward documents that are closer to its ideal binary pattern, while avoiding unnecessary full-score calculations?

1.3 Exploring new methods

From the previous section, we know two useful things before we even start searching:

We know what the ideal binary document looks like from the signs of the query. We know that some bits matter more than others, based on the magnitude $|q_i|$.

The next question is therefore:

Can we build an index that uses this information directly, instead of treating every document inside the selected clusters equally?

This led me to explore two approaches.

The first, Bitplanes, starts from a large set of candidate documents and progressively narrows it down using the query.

The second, Backward Walk, approaches the problem from the opposite direction: it starts from a very specific binary pattern and progressively relaxes the constraints until enough useful candidates are found.

1.3.1 Bitplanes

Now that we know what the ideal document looks like based on the query, one idea is to build a data structure that makes filtering by individual bits very cheap.

Normally, binary documents are stored row by row:

ID 2: 10100
 ID 3: 11001
 ID 4: 11000

Instead, we can additionally store an inverted representation of the same bits:

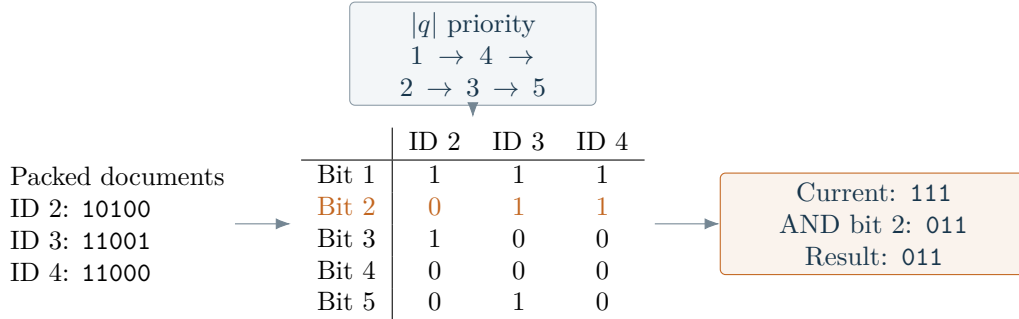


Figure 2: The same document signs, stored in an extra layout for bitwise filtering.

Each row above is what I call a bitplane: one bit position across many documents.

Suppose the query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the magnitude of the query values, we can prioritize the bits approximately as

$$1, 4, 2, 3, 5,$$

because

$$0.7 > 0.6 > 0.5 > 0.4 > 0.3.$$

Now suppose our current candidate mask is

$$111$$

meaning that IDs 2, 3 and 4 are all still possible candidates.

For bit 2, the query prefers 1. The corresponding bitplane is

$$011$$

so keeping the documents that match the query is simply

$$111 \& 011 = 011.$$

We immediately reduce the candidate set to IDs 3 and 4.

This is attractive because the filtering itself consists mostly of bitwise operations, which modern CPUs can perform very efficiently.

A simplified first version of the idea might therefore look like:

```

def full_score(query, code):
    return sum(q if bit else -q for q, bit in zip(query, code))

def greedy_search(query, cluster, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    active = cluster.full_mask
    for bit in order:
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        active &= preferred
        if active.bit_count() <= leaf_size:
            break

    best = TopK(k)
    for row_id, code in cluster.rows_in_mask(active):
        best.add(row_id, full_score(query, code))
    return best.results()

```

However, this simple version has a major problem.

PROBLEM 1: Greedily following the preferred bits can miss the best document

Consider:

$$q = [0.6, 0.5, 0.4, 0.3].$$

The ideal binary document is 1111.

Suppose the database only contains:

Document A: 0111

Document B: 1000

If we greedily start with the first bit, the query prefers 1. We would therefore keep Document B and throw away Document A.

But their complete scores are:

$$S(A) = -0.6 + 0.5 + 0.4 + 0.3 = 0.6$$

while

$$S(B) = 0.6 - 0.5 - 0.4 - 0.3 = -0.6.$$

So Document A is actually better.

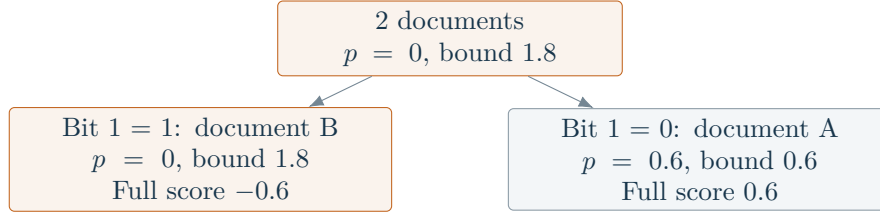
The problem is that matching one important bit does not guarantee that the remaining bits will also match.

Solution: branching

Instead of discarding the alternative, we keep both branches.

The branch matching the preferred bit receives no additional mismatch penalty. The alternative branch is still kept, but its penalty increases by $|q_i|$.

We then explore the branches in order of smallest accumulated penalty. This gives us a recursive search tree rather than one greedy path.



After scoring B, the other bound is $0.6 > -0.6$.
We must explore A; the first choice was not enough.

Figure 3: Keep the alternative until its score bound proves it cannot improve the result.

Stopping condition

Recall:

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|.$$

If a branch has already accumulated mismatch penalty p , any document below it must have penalty at least p .

Therefore,

$$U_{\text{branch}} = Q - 2p$$

is an upper bound on the score of every document inside that branch.

For example, suppose $Q = 2.5$ and we are looking for the top two results. We have already found 2.5 and 1.9, where 1.9 is the current top-2 threshold.

Now consider an unexplored branch with $p = 0.5$. Its absolute best possible score is

$$2.5 - 2(0.5) = 1.5.$$

Since $1.5 < 1.9$, nothing inside that branch can enter our current top two. We can safely discard the entire subtree.

So:

$$Q - 2p < \text{current } K\text{th score} \Rightarrow \text{prune branch}$$

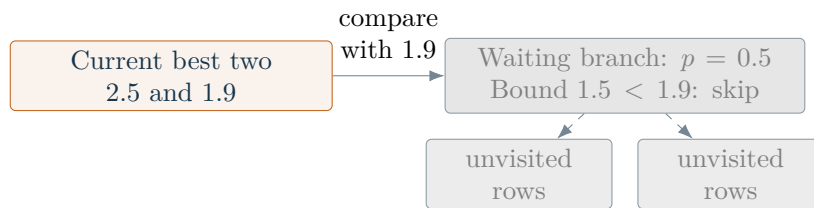


Figure 4: The whole gray subtree can be skipped because even its best possible score is too low.

If the best remaining branch in the queue already fails this test, then every other branch is worse and the entire search can stop.

The resulting algorithm is approximately:

```

from heapq import heappop, heappush
from itertools import count

def bitplane_search(query, clusters, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    ideal_score = sum(abs(q) for q in query)
    sequence = count()
    queue = []
    for cluster in clusters:
        if cluster.full_mask:
            heappush(queue, (0, next(sequence), 0,
                             cluster.full_mask, cluster))

    best = TopK(k)
    while queue:
        penalty, _, depth, active, cluster = heappop(queue)
        upper_bound = ideal_score - 2 * penalty
        if best.full() and upper_bound < best.worst_score():
            break
        if active.bit_count() <= leaf_size or depth == len(query):
            for row_id, code in cluster.rows_in_mask(active):
                best.add(row_id, full_score(query, code))
            continue

        bit = order[depth]
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        matching = active & preferred
        # Keep the other half too; its remaining bits may be better.
        children = ((matching, 0), (active ^ matching, abs(query[bit])))
        for mask, extra_penalty in children:
            if mask:
                heappush(queue, (penalty + extra_penalty,
                                next(sequence), depth + 1, mask, cluster))

    return best.results()

```

This is essentially the branch-and-bound Bitplane search used in the current experiments.

PROBLEM 2: Bitwise operations are cheap, but the bitmap itself can still be large

There is another issue.

A bitwise AND sounds almost free, but it is not literally one operation when the cluster contains thousands or millions of documents.

Suppose one cluster contains 6400 documents. With 64-bit machine words, one bitplane contains

$$6400/64 = 100$$

machine words.

So one split roughly requires us to loop over those 100 words.

Even worse, suppose after several splits only 80 documents remain. The physical mask can still be 100 words wide. So the next split may still need to process those same 100 words.

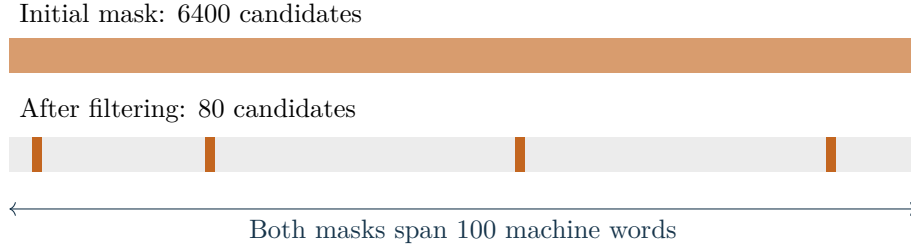


Figure 5: Fewer active document positions do not make this dense mask physically shorter.

Bitmap traversal cost. In general, for a cluster containing n documents, one dense bitplane operation costs roughly $O(\lceil n/64 \rceil)$ word operations. If we visit J split nodes, this becomes approximately $O(J \lceil n/64 \rceil)$, before even counting queue operations and the final document scoring. This was one of the important findings from the earlier experiments: reducing the number of fully scored documents does not automatically mean reducing total query time, because the bitmap traversal itself can become expensive.

Solution 1: Parameter tuning. This problem is technically something we can optimize. For example, we can tune cluster size during coarse routing; number of clusters probed; how deep we split; leaf size; when we switch back to ordinary full scoring. So Bitplanes is not fundamentally broken. The question is finding the point where scores avoided are worth more than additional bitmap traversal.

Solution 2: Other approach. However, thinking about this cost led me to a second idea. Bitplanes starts with a large set of documents and repeatedly narrows it down. What if we instead did the opposite? If Bitplanes can be thought of as bottom-up, then Backward Walk can be thought of as top-down: start from the most specific binary pattern we can describe and progressively broaden the search only when necessary.

1.3.2 Backward Walk

Backward Walk builds on the same observation:

Given a query, we already know what its ideal binary document should look like.

However, instead of starting with thousands of documents and filtering them using bitplanes, we try to start directly from a small bucket of documents that already looks similar to the query.

Suppose the binary query begins with 101101001011....

Using all 256 bits as a lookup key would be far too specific. With a large binary space, the probability that an exact 256-bit pattern actually exists in the database is extremely small.

Instead, suppose we choose a prefix length x . For example, the first 16 bits:

1011010010110101

During index construction, documents sharing this prefix are placed in the same bucket or contiguous range.

At query time, we directly look up this prefix. If it contains enough useful documents, we score them using the complete 256-bit score. If it is empty, or contains too few candidates, we remove one constraint:

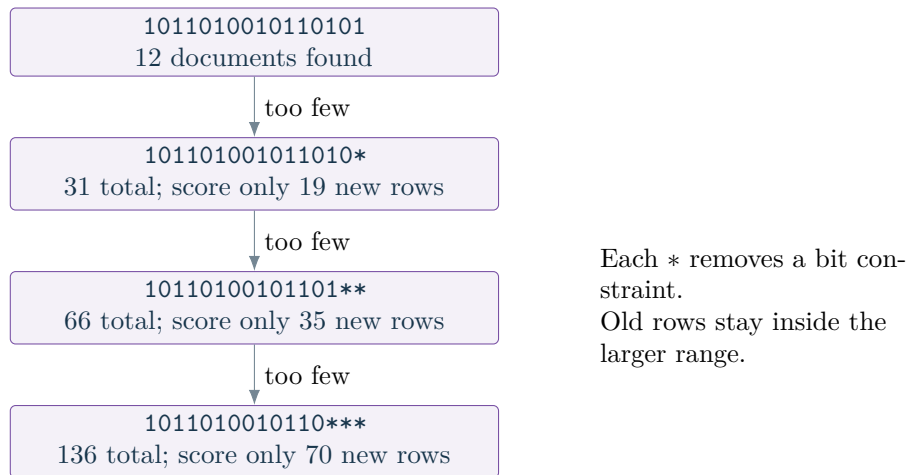


Figure 6: Illustrative counts: $12 + 19 + 35 + 70 = 136$ distinct rows scored, not $12 + 31 + 66 + 136$.

Every step moves to a larger candidate set.

This is why I call the method Backward Walk: instead of progressively adding constraints to a large candidate set, we begin from a very specific location and progressively walk backward toward more general prefixes.

Note that this bit prefix already has an exponential property: if the bits are independent and equally likely to be 0 or 1, each removed bit doubles the expected number of matching documents.

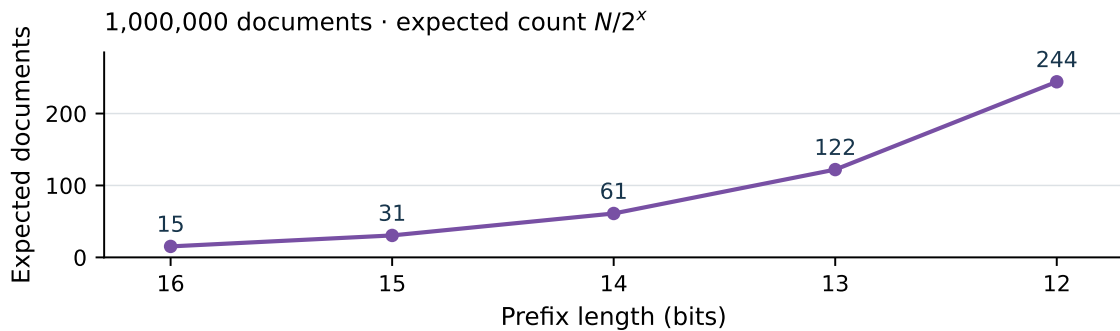


Figure 7: Expected counts for one global cluster; labels are rounded. This estimates rows, not recall.

With a trie or documents ordered by their prefix, the previous range is contained inside the parent range. We only need to process the newly introduced documents.

A simplified query algorithm becomes:

```

def backward_walk(query, clusters, k, start_depth, candidate_target):
    signs = tuple(q >= 0 for q in query)
    previous = [None] * len(clusters)
    best = TopK(k)
    scored = 0
    for depth in range(start_depth, -1, -1):
        for index, cluster in enumerate(clusters):
            left, right = cluster.prefix_range(signs[:depth])
            if previous[index] is None:
                new_ranges = [(left, right)]
            else:
                old_left, old_right = previous[index]
                # Only the new rows beside the old range need scores.
                new_ranges = [(left, old_left), (old_right, right)]
            for start, stop in new_ranges:
                for row_id, code in cluster.rows_in_range(start, stop):
                    best.add(row_id, full_score(query, code))
                scored += stop - start
            previous[index] = (left, right)

        # Finish this depth in every cluster before testing the target.
        if candidate_target > 0 and scored >= candidate_target:
            break
    return best.results()

```

There is one important distinction here.

Stopping simply because we have collected, for example, 100 candidates is an approximate stopping rule. It does not mathematically prove that a better document does not exist outside the current prefix.

Therefore, in the experiments, recall still needs to be measured against the same exact reference results. An exact version would require a valid upper bound on the scores of documents outside the currently visited prefix.

Relationship to HNSW

At first glance, this approach may sound somewhat similar to HNSW because both methods use a hierarchy to progressively change the search space.

However, the hierarchy is being used differently.

HNSW builds a proximity graph. During a query, it begins from sparse upper layers, navigates toward promising neighbors, and progressively descends toward the denser bottom layer [2].

Backward Walk does not need to navigate between neighboring document vectors. The query itself directly tells us where to begin:

query → binary prefix → exact prefix lookup

We then broaden that prefix only when necessary.

So conceptually:

	HNSW	Backward Walk
Index	Proximity graph	Prefix / key hierarchy
Query start	Sparse graph layer	Query's own binary prefix
Movement	Neighbor → neighbor	Specific prefix → broader prefix
Index insertion	Find neighbors + maintain edges	Compute prefix + insert into bucket/range
Search signal	Vector distance	Prefix agreement

Potential advantage 1: cheaper index construction and updates

One possible advantage that I would like to investigate further is the cost of adding new documents. For HNSW, inserting a new vector generally requires searching the existing graph to find suitable neighbors and then creating or updating graph connections.

For Backward Walk, the conceptual insertion process is much simpler:

document $\rightarrow$ embedding $\rightarrow$ binary signs $\rightarrow$ prefix key $\rightarrow$ corresponding bucket

There is no nearest-neighbor graph that needs to be constructed for each new document. So Backward Walk may be attractive for a search index that changes frequently.

I would frame this as a hypothesis to verify, because the actual insertion cost still depends on how the buckets, ranges and on-disk/in-memory index are implemented.

Potential advantage 2: Matryoshka prefixes give the relaxation some meaning

There is also a useful connection with Matryoshka embeddings.

A Matryoshka model is trained so that shorter prefixes of the embedding remain useful representations [3].

Therefore, moving from, for example, 16 bits $\rightarrow$ 15 bits $\rightarrow$ 14 bits is not simply dropping arbitrary dimensions from an ordinary embedding. We are progressively moving to shorter prefixes of a representation that was specifically trained to retain useful information at multiple prefix lengths.

However, this does not mean that the first coordinate is always more important than the second, or that the nearest neighbor under 16 dimensions must remain the nearest neighbor under 15 dimensions.

Matryoshka training gives us a reason to believe that shorter prefixes remain meaningful; the actual recall of each prefix still has to be measured.

2 Preliminary calculations

2.1 Time complexity

Alright, so far we have three methods. We can first break down the time complexity of each method to clearly see how the different parameters, or hyperparameters, can influence the overall latency.

The goal is to find a mathematically optimized set of parameters for each method. In the end, we will run experiments to verify these predictions and determine whether Method B and Method C can actually outperform Method A in some, or possibly all, cases.

To make the comparison easier to follow, let's use the same small example for all three methods. Suppose we open one cluster containing three documents and want to return the best two results:

$$N = 3, \quad C = P = 1, \quad d = 5, \quad K = 2.$$

Here N is the total number of documents, C the number of clusters, P how many clusters we open, d the number of bits per document, and K how many results we keep. The query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{ideal signs: } 11001.$$

ID	Document bits	Full score	Calculation
0	10100	0.1	$0.7 - 0.5 - 0.4 + 0.6 - 0.3$
1	11001	2.5	$0.7 + 0.5 + 0.4 + 0.6 + 0.3$
2	11000	1.9	$0.7 + 0.5 + 0.4 + 0.6 - 0.3$

Bits are shown in their original order, left to right. These rounded scores are a small worked example; the later experiments use the full datasets. We use F_A , F_B and F_C to count how many documents each method fully scores.

For the larger formulas, let $n = N/C$ be the average number of documents per cluster. If clusters are approximately balanced, opening P of them exposes about $Pn = PN/C$ documents. Actual clusters can be uneven, so the experiments also record their exact sizes.

2.1.1 Original method

The original method is relatively straightforward. After selecting the clusters, every document inside those clusters is fully scored.

Step 1: open the clusters and prepare the query.

For our example, we open cluster 0: [ID 0, ID 1, ID 2]. Following the four-sign lookup-table scoring described by Exa [1], we divide the query into

$$[0.7, 0.5, -0.4, -0.6] \quad \text{and} \quad [0.3, 0, 0, 0].$$

Each group has $2^4 = 16$ possible sign patterns, so two groups contain $2 \times 16 = 32$ table entries. The last group is padded with zeros. Our implementation checks four positions per entry: $32 \times 4 = 128$ position checks during setup. Since four is fixed, building these tables costs $O(d)$.

One full document score then needs

$$G = \lceil d/4 \rceil$$

lookup-table terms. Here $G = 2$.



Figure 8: After routing, every selected document reaches A’s full scorer.

Step 2: score each document.

Each of our three documents reads two score-table entries:

$$\text{ID 0} \longrightarrow 0.1, \quad \text{ID 1} \longrightarrow 2.5, \quad \text{ID 2} \longrightarrow 1.9.$$

That gives $3 \times 2 = 6$ score-table terms. For F_A documents, the work is $F_A \lceil d/4 \rceil$ terms. With balanced clusters,

$$F_A \approx \frac{PN}{C}, \quad T_{\text{score},A} = O\left(\frac{PN}{C}d\right).$$

For fixed d , scoring is linear in the number of selected documents.

Step 3: keep the best K .

While scoring the documents, we keep only the best two:

$$[0:0.1] \longrightarrow [1:2.5, 0:0.1] \longrightarrow [1:2.5, 2:1.9].$$

The display is sorted for readability. The code uses a bounded heap, which keeps the worst retained result ready to replace. Updating it takes at most $O(F_A \log(K+1))$ work, and sorting the final results takes $O(K \log(K+1))$. The $+1$ also covers the constant work of keeping just one result.

Total.

In this example, we open one cluster, build 32 table entries, read six score-table terms and offer three scores to the result heap. These are different kinds of work; we cannot add them as equal CPU operations.

Let T_0 include routing, common query preparation and final result sorting. Let a_A be the average time to fully process one selected document, including its score and result update. Then

$$T_A \approx T_0 + \frac{PN}{C} a_A.$$

For uneven clusters, replace PN/C by the actual number of opened rows, $F_A = \sum_{c \in \mathcal{O}} n_c$. Here $\mathcal{O}$ is the set of opened clusters and n_c is the size of cluster c .

The main parameters are C and P . Increasing C makes the average cluster smaller, but may require more probes to maintain recall. Increasing P exposes more documents. We therefore need the combination that gives the lowest time while meeting the recall target. T_0 also changes with routing settings; it is not a fixed number across every configuration.

2.1.2 Bitplanes

Bitplanes try to reduce the number of documents that reach the full scorer. The tradeoff is that we now spend additional work traversing the bitplanes and maintaining alternative branches. For the same three documents, choose a leaf size of two and leave the work budget unlimited.

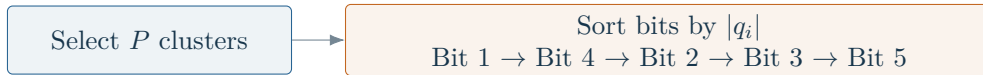
Step 1: open, prepare and order the bits.

We begin with the same routing and 32-entry score table as A. The additional step is to sort the query coordinates by decreasing absolute value:

$$1 (0.7) \rightarrow 4 (0.6) \rightarrow 2 (0.5) \rightarrow 3 (0.4) \rightarrow 5 (0.3).$$

Sorting costs $O(d \log d)$. We also add the absolute values, which takes $O(d)$ work:

$$Q = \sum_{i=1}^d |q_i| = 2.5.$$



Bitplanes add a search stage inside the selected clusters. They do not replace routing.

Step 2: split the candidate masks.

A mask has one position per document. With 64-bit words, a cluster containing n rows needs

$$W = \lceil n/64 \rceil$$

words per mask. Our three rows fit in one word, so $W = 1$. The starting mask is 111: all three rows are active.

At bit 1, every document matches the preferred sign, so $111 \text{ AND } 111 = 111$. At bit 4, every document again matches the preferred sign, zero. At bit 2, the plane is 011, so the preferred branch becomes $111 \text{ AND } 011 = 011$. It contains IDs 1 and 2. The alternative is 100, containing ID 0 with penalty $p = |q_2| = 0.5$.

Three splits visit $3 \times 1 = 3$ word positions. Each visit reads the mask and a bitplane, forms and writes the two children, and counts matching bits. One word visit therefore contains several CPU operations.

The physical mask does not shrink when fewer documents remain. For example, a 6,400-document cluster needs 100 words; a mask with only 80 surviving documents still has those same 100 words. If we average S splits per opened cluster, the split work is proportional to

$$PS \lceil N/(64C) \rceil.$$

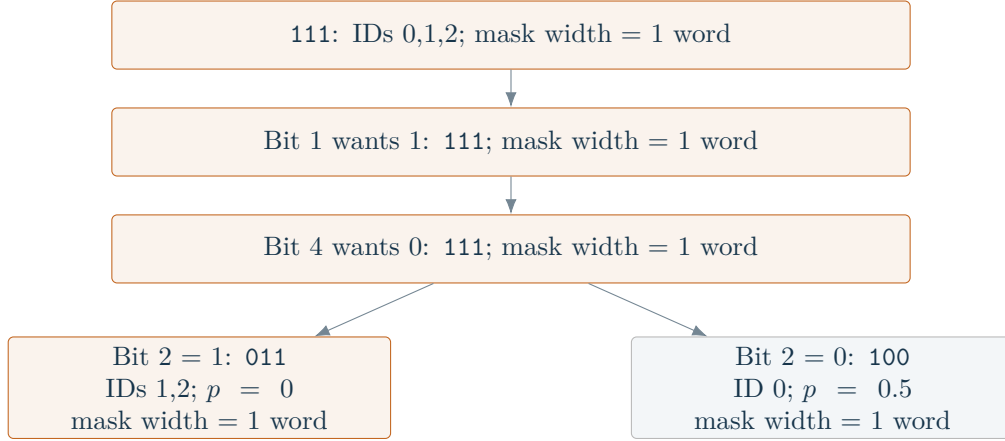


Figure 9: The first two splits remove nothing. The third keeps IDs 1 and 2 on the preferred branch, with ID 0 waiting as an alternative.

We also write the initial masks once. With actual cluster widths W_c , that adds $\sum_{c \in \mathcal{O}} W_c$ word writes.

Step 3: score the surviving rows.

The preferred mask 011 contains two documents, so it fits the leaf limit. We read its one mask word and fully score IDs 1 and 2. The two scores use four table terms and produce 2.5 and 1.9.

Let F_B count the documents that B fully scores, and let

$$f_B = \frac{F_B}{F_A}$$

be the fraction of opened documents that reaches the scorer. In this calculation, A and B open the same clusters, so F_A is the scan cost for that routing choice. When we later choose different routing settings per method, we compare their own row counts.

With balanced clusters, $F_B \approx f_B P N / C$, giving the time term

$$f_B \frac{P N}{C} a_B.$$

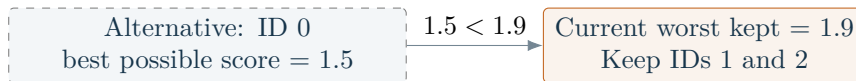
a_B includes scoring and result updates. Here $F_B = 2$ and $f_B = 2/3$. The goal is $f_B < 1$, so we avoid some full scores. Reading leaf masks still takes work: if H_c leaves are scored in cluster c , those reads visit $\sum_{c \in \mathcal{O}} H_c W_c$ words.

Step 4: check the waiting branches.

ID 0's branch has penalty $p = 0.5$. Its best possible score is

$$Q - 2p = 2.5 - 2(0.5) = 1.5.$$

We already have scores 2.5 and 1.9. Since $1.5 < 1.9$, the waiting branch cannot improve our top two, and we discard it without scoring ID 0.



A bound check takes $O(1)$ work once Q and the penalty are known. Managing the waiting sets also costs time. If V sets are processed, including split nodes and leaves, queue work is bounded by $O((P + V) \log(P + V + 1))$. Here $V = 4$: three splits and one leaf.

Total.

We visit three split-word positions and one leaf-mask word, then calculate two full scores instead of three. We also sort the query positions, initialize masks and manage the waiting branches. Let b be time per split-word visit and $T_{\text{other},B}$ cover those other tasks. Then

$$T_B \approx T_0 + PS \left\lceil \frac{N}{64C} \right\rceil b + f_B \frac{PN}{C} a_B + T_{\text{other},B}.$$

For uneven clusters, use $b \sum_{c \in \mathcal{O}} J_c W_c$ for splitting, where J_c is the actual split count in cluster c , and $F_B a_B$ for scoring. The condition we want is

$$\text{cost of scores avoided} > \text{additional bitmap and branching cost.}$$

A smaller leaf size can reduce f_B , but usually increases S . Larger clusters make each bitmap wider. Cluster count, probes, leaf size and node budget therefore need to be chosen together. Fewer surviving documents alone do not prove lower latency.

2.1.3 Backward Walk

Backward Walk changes the direction of the search. Instead of starting from many documents and progressively removing them, we start from a specific binary prefix and broaden the candidate set.

For the same example, store $h = 5$ prefix bits, start at depth $x = 5$, and stop after a complete depth has produced at least two candidates. Here h is the stored key width and x is how many of those bits we initially require.

Step 1: look up the full prefix.

Before queries arrive, we store the documents in sorted key order:

10100 (ID 0), 11000 (ID 2), 11001 (ID 1).

At query time, we build the common score table and read h query signs once to form 11001, taking $O(h)$ work. Later depths reuse this key. Two binary searches find the beginning and end of its matching range, $[2, 3)$, which contains ID 1. We fully score that row and obtain 2.5. Only one candidate has been found, so we continue.

Step 2: remove one prefix constraint.

The next prefix is 1100*. Its range, $[1, 3)$, contains IDs 2 and 1. ID 1 was already scored, so we score only the new slice $[1, 2)$, containing ID 2. Its score is 1.9.

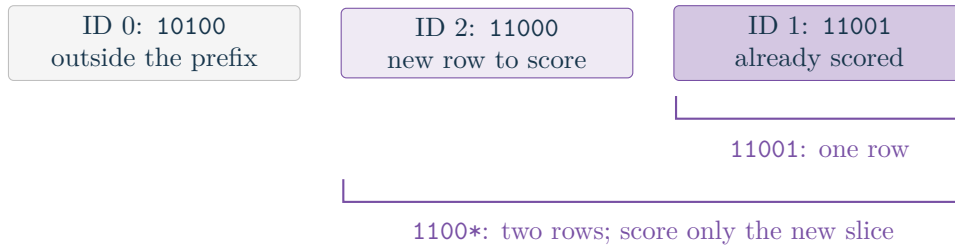


Figure 10: The wider prefix includes the old range. Only ID 2 adds a new full score.

We have now scored two distinct candidates, so the candidate target is reached and the search stops. This returns the true top two in our example. Collecting K candidates alone does not guarantee exact top- K recall on other data; the later experiment measures how often this stopping rule recovers the reference IDs.

Step 3: count the prefix lookups.

If we start at depth x and stop at depth ℓ , the number of visited depths is

$$L = x - \ell + 1.$$

At every nonzero depth, each cluster needs two binary searches. Depth zero already has a known range, the entire cluster, so it needs none. Let L_+ count only the nonzero depths: $L_+ = L$ when $\ell > 0$, and $L_+ = L - 1$ when $\ell = 0$. The number of binary-search calls is therefore

$$2PL_+.$$

Each call takes $O(\log(n+1))$ comparisons for a cluster with n rows. For balanced clusters, the lookup work is approximately proportional to $2PL_+ \log_2(n+1)$.

In our example, $P = 1$, $L = L_+ = 2$, so we make four binary-search calls. Each uses at most two comparisons on three rows, giving at most eight key comparisons, plus the range checks.

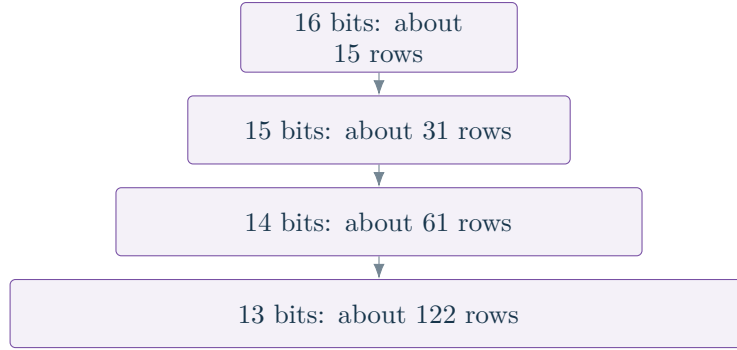
Step 4: count the documents scored.

Let F_C be the number of distinct documents exposed when the walk stops. Since each wider range contains the previous one, we score each document once. Full scoring and result updates therefore contribute $F_C a_C$, where a_C is the average time per processed row. Our example scores two rows and reads four score-table terms.

To estimate how quickly the set grows, suppose bits are independent and equally likely to be zero or one *within the opened clusters*. For a fixed prefix depth ℓ , each row matches with probability $2^{-\ell}$. In one cluster of size n , the expected count is $n/2^\ell$; across P equal-sized clusters,

$$\mathbb{E}[F_C \mid \text{fixed depth } \ell] \approx \frac{PN}{C2^\ell}.$$

For a flat collection of one million documents, this gives:



Removing one bit doubles the expected count under this model. The box widths illustrate growth, rather than an exact scale. Real embedding bits can be correlated or uneven, especially after clustering. This fixed-depth count is not a recall estimate, and it does not predict the count at a depth chosen adaptively by the search.

Total.

Let t_b be average time per binary-search comparison, T_{key} the one-time query-key work, and T_{ranges} the range updates and new-slice bookkeeping. Then

$$T_C \approx T_0 + T_{\text{key}} + 2PL_+ \log_2\left(\frac{N}{C} + 1\right) t_b + F_C a_C + T_{\text{ranges}}.$$

For uneven clusters, replace the lookup term by $2L_+ t_b \sum_{c \in \mathcal{O}} \log_2(n_c + 1)$. Range bookkeeping costs $O(PL)$, scoring costs $O(F_C d)$, and result updates take up to $O(F_C \log(K+1))$.

The lookup itself grows only logarithmically with cluster size. The main question is how many

documents are exposed before we stop. If a deep prefix already contains useful candidates, $F_C \ll PN/C$, and C may avoid most of A 's scoring. If we have to reach depth zero, $F_C = F_A$, so C performs all of A 's scoring plus its prefix work.

Section 3.6 gives an optional score bound that can prove the answer before the root. Its query-weight setup and checks are included in T_{key} and T_{ranges} . A different index, such as a trie, would change the lookup term; the experiments here use sorted keys.

2.1.4 Parameters

Method	Main parameters	Main latency tradeoff
Original method	Clusters C , probes P	Routing and full scores versus recall
Bitplanes	C , P , leaf size, node budget	Bitmap traversal versus scores avoided
Backward Walk	C , P , starting depth x , stopping rule	Prefix lookups versus documents exposed

The methods pay for different kinds of work. A mainly pays for full document scores. B adds bitmap traversal to avoid some of those scores, while C adds prefix lookups to start from a smaller set of documents. We use these expressions to choose parameter ranges, then check their latency at the same recall requirement and memory limit. The time per operation must be measured: an index that fits in the CPU cache can behave differently from one read from RAM.

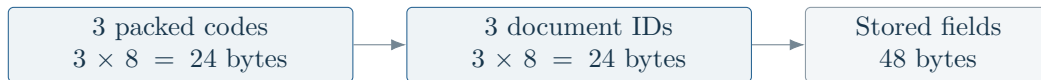
2.2 Memory complexity

We keep N documents of d bits in C clusters, opening P per query. Cluster c has n_c rows and $W_c = \lceil n_c/64 \rceil$ words per mask. In bytes, a packed word or ID uses eight, a float query value four, and a double-precision score eight. Subscripts on M and Q name the stored and query fields being counted.

For the example, $N = 3$, $d = 5$, $C = P = 1$ and $K = 2$. We count named fields first; unused capacity, overlapping buffers and the rest of the process need separate measurements.

2.2.1 Original method

Step 1: store the codes and IDs before search.



Each five-bit row uses one eight-byte word: $M_{\text{rows}} = N(8 \lceil d/64 \rceil + 8)$ bytes, or $O(N \lceil d/64 \rceil + N)$. Routing adds M_{routing} bytes. If the router reads d_R coordinates, its C float32 centroids and eight-byte labels use $4Cd_R + 8C$ bytes. The one-cluster example needs no centroid search.

Step 2: hold the query, score table and current results during search.

$$\boxed{5 \times 4 = 20} \quad + \quad \boxed{32 \times 8 = 256} \quad + \quad \boxed{2 \times (8 + 8) = 32} \quad = \quad 308 \text{ bytes.}$$

Left to right are the query, score table and one buffer of result IDs and scores. Their field count is $Q_0 = 4d + 128 \lceil d/4 \rceil + 16K$, or $O(d + K)$. The heap, final sorting and Python output can overlap in memory; Q_0 counts one result buffer, not the complete peak.

Total counted fields: 48 bytes stored + 308 query bytes = 356 bytes. This is not total process RAM; cluster records and allocation overhead remain additional.

2.2.2 Bitplanes

Step 1: keep A's rows and add the bitplanes.

	ID 0	ID 1	ID 2	Packed storage
Bit 1	1	1	1	8 bytes
Bit 2	0	1	1	8 bytes
Bit 3	1	0	0	8 bytes
Bit 4	0	0	0	8 bytes
Bit 5	0	1	0	8 bytes

Each plane has three useful bits but occupies one word. Five planes add $5 \times 8 = 40$ bytes. Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{planes}} = 8d \sum_c \lceil n_c/64 \rceil$ bytes. Include padding in every cluster.

Step 2: hold masks while splitting.

$$\underbrace{111}_{\text{parent: 8 bytes}} \longrightarrow \underbrace{011}_{\text{child: 8 bytes}} + \underbrace{100}_{\text{child: 8 bytes}}.$$

The parent and both full-width children coexist: $3 \times 8 = 24$ live mask bytes. Five sorted positions add $5 \times 8 = 40$ bytes on a 64-bit machine, giving $308 + 24 + 40 = 372$ bytes for these query fields. Queue records are extra. Let $A_c(t)$ count live masks in cluster c at time t , including waiting branches and the parent being split. Their peak storage is $M_{\text{masks}} = 8 \max_t \sum_{c \in \mathcal{O}} A_c(t) W_c$.

Total counted fields: 88 bytes stored + 372 query bytes = 460 bytes. Stored size is $O(N \lceil d/64 \rceil + N + d \sum_c W_c)$; query masks grow with the number and width of live branches, not just the number of remaining document IDs.

2.2.3 Backward Walk**Step 1: store rows in prefix order.**

Position	Full binary document	ID
0	10100	0
1	11000	2
2	11001	1

The full codes and IDs use $3 \times (8 + 8) = 48$ bytes. Three four-byte prefix keys add 12 bytes, totaling 60. The key type supports stored widths up to 32 bits. Every prefix range uses this same sorted array, without a separate row list per depth.

$$M_{C,\text{fields}} = N(8 \lceil d/64 \rceil + 8 + 4) + M_{\text{routing}}.$$

This counts the fields used by the index. Its allocated arrays can be larger because capacity is rounded up, and construction can hold temporary copies alongside the finished arrays.

Step 2: hold range boundaries during search.

Each selected-cluster record contains an eight-byte cluster pointer and two eight-byte offsets for its previous range: $(8 + 8 + 8)P = 24P$ bytes on a 64-bit machine. The boundaries alone account for $16P$. The query key uses four more bytes, so the example adds $24 + 4 = 28$ bytes to A's common query fields.

The previous range tells us which rows were already scored, so we need neither a bitmap over all documents nor a list of visited IDs.

Total counted fields: 60 bytes stored plus $308 + 24 + 4 = 336$ query bytes gives 396 bytes for this example, before capacity, local variables and overlapping buffers. Persistent space is

$O(N \lceil d/64 \rceil + N)$ plus routing, and query space is $O(P + d + K)$. The Experiment section reports process memory separately, including the peak during construction.

2.3 Recall estimation

If the reference IDs are $[1, 2]$ and we return $[1, 0]$, we recover one of the two required IDs: recall is $1/2 = 50\%$. Returning $[1, 2]$, as all three worked searches above do, gives $2/2 = 100\%$. We find the reference by scoring every document with the full binary score and using the same document-ID rule to resolve ties.

Let K be the number of reference results per query and n_q the number of queries. For query q , G_q is the set of reference IDs and $O_{a,q}$ is the set returned by method a , where a is A, B or C. The intersection contains the IDs shared by both sets, and the bars count them. Across the full cached query set, average recall is

$$\overline{R}_a = \frac{1}{n_q K} \sum_q |G_q \cap O_{a,q}|.$$

2.3.1 Original method

Let g_q count reference IDs in the opened clusters. A scores every opened row using the reference score and tie order, so all g_q of those IDs remain in its local top K . Its recall is g_q/K .

2.3.2 Bitplanes

Suppose eight of ten reference IDs are in the opened clusters, and B returns six of those eight. Recall is $(8/10)(6/8) = 60\%$. We call the fraction recovered within the opened clusters $r_{B,q}$, giving recall $(g_q/K)r_{B,q}$. If $g_q = 0$, recall is zero. Unlimited local search with safe bounds recovers every opened reference ID.

2.3.3 Backward Walk

Let $D_{q,\ell}$ be the rows in the opened clusters matching the query's first ℓ bits. After completing depth ℓ across those clusters, every row in this set has been scored once. Keeping its full-score top K gives recall $|G_q \cap D_{q,\ell}|/K$. We need the intersection with the reference IDs to calculate that fraction; knowing how many rows match the prefix does not tell us how many reference results they contain.

Consider $q = [0.1, 0.9]$ and two documents, 10 and 01. The ideal pattern is 11, but it is absent. If we shorten it to $1*$, we find 10, whose score is $0.1 - 0.9 = -0.8$. The other document, 01, scores $-0.1 + 0.9 = 0.8$ and is actually better. Stopping after finding one candidate would therefore give zero top-one recall. We must widen to depth zero to reach that other document. The fixed prefix order removed the more important second bit first; Matryoshka training does not sort each query's values by magnitude.

At depth zero, $D_{q,0}$ contains every opened row, so C matches A's recall for the same routing choice. With the default stopping rule, a zero candidate target reaches this depth. The score check in Section 3.6 can prove the same result sooner. A positive target can still stop earlier and miss reference IDs, because a candidate count does not bound the scores of unseen documents. Opening every cluster and completing exact local search gives an exact search over the whole collection.

A simple approximation. Suppose 80% of the reference documents agree with the query's first sign, and 75% agree with its second sign. If the two agreements are independent, about $0.8 \times 0.75 = 60\%$ agree with both. The multiplication is an assumption about those events; it

does not follow from the length of the embedding.

For a fixed ending depth ℓ , let p_i be the fraction of opened reference documents that agree at position i , counted across queries. Let $\bar{R}_A$ be the fraction of all reference IDs whose clusters we opened. Under independent sign agreement, we would estimate

$$\hat{R}_C(\ell) = \bar{R}_A \prod_{i=1}^{\ell} p_i.$$

The hat marks an estimate. We can obtain the p_i values from a sample of queries and their exact reference documents, then compare this product with the fraction that actually matches the whole prefix. For one global cluster, $\bar{R}_A = 1$. For clustered search, the rates must describe reference documents in the opened clusters.

This calculation assumes every query ends at the same depth. In the actual walk, the candidate target decides when each query stops. We therefore check reference membership at each query’s actual ending prefix. Substituting the average ending depth into the product would ignore how document counts and sign matches vary together. The experiment checks the simpler fixed-depth approximation separately.

2.4 Summary

Time complexity

Method	Approximate query time, using balanced cluster sizes
Original method	$T_A \approx T_0 + \frac{PN}{C} a_A$
Bitplanes	$T_B \approx T_0 + PS \left\lceil \frac{N}{64C} \right\rceil b + f_B \frac{PN}{C} a_B + T_{\text{other},B}$
Backward Walk	$T_C \approx T_0 + T_{\text{key}} + 2PL_+ \log_2(n+1)t_b + F_C a_C + T_{\text{ranges}}$

Memory complexity

Method	Stored index fields	Temporary query fields
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + M_{\text{prefix}}$	$Q_0 + Q_C$

Recall

Method	Per-query recall	Exactness within opened clusters
Original method	$\frac{g_q}{K}$	Score all opened documents
Bitplanes	$\frac{g_q}{K} r_{B,q}$	Complete search using safe score bounds
Backward Walk	$\frac{ G_q \cap D_{q,\ell} }{K}$	Reach depth zero, or prove the score bound

Variable definitions

Symbol	Definition
N	Total documents.
C, P	Total clusters, and how many we open for one query.
$n = N/C$	Average rows per cluster; the time table assumes opened clusters are about this size.
K, d	Results requested, and bits per document.
T_0	Routing, common query preparation and final sorting. Its value changes with the chosen settings.

Symbol	Definition (continued)
a_A, a_B, a_C	Average time to fully score a document and update the results in each method.
S, b	Average splits per opened cluster, and time per split-word visit. One visit contains several CPU operations.
$f_B = F_B/F_A$	Fraction of opened documents that B scores. F_A counts a full scan of those same clusters, not A’s separately tuned setting.
$T_{\text{other},B}$	Query ordering, weight sums, initial masks, queue work and leaf-mask reads.
h, x, ℓ	Stored prefix width, starting depth and final depth. $0 \leq x \leq h \leq \min(d, 32)$ in this layout.
$L = x - \ell + 1$	How many prefix depths we visit, including depth zero if reached.
L_+	How many visited depths are nonzero: L if $\ell > 0$, otherwise $L - 1$. Only these need binary searches.
t_b	Average time per key comparison during a binary search.
T_{key}	Build the query key once, plus query-weight setup if the exact-stop option is used.
F_C	Distinct documents scored by Backward Walk. Previously scored rows are not scored again.
T_{ranges}	Range updates and new-slice checks at each depth, plus optional score-bound checks. The binary searches are counted separately.
M_0	Packed codes, IDs and router fields: $N(8 \lceil d/64 \rceil + 8) + M_{\text{routing}}$ bytes. This is the same field layout, not necessarily the same byte total when cluster settings differ.
M_{routing}	Router fields; $4Cd_R + 8C$ bytes for float32 centroids and 64-bit labels, where d_R is the routing dimension.
M_{planes}	Extra bitplane fields: $8d \sum_{c=1}^C \lceil n_c/64 \rceil$ bytes, where n_c is cluster c ’s row count. Include padding per cluster.
M_{prefix}	Four-byte keys: $4N$ bytes. The same sorted array serves every prefix; there is no stored row list per depth.
Q_0	Query, score table and one result buffer: $4d + 128 \lceil d/4 \rceil + 16K$ bytes.
Q_B	Bit ordering, live branch masks and queue records. Each live mask keeps its full cluster width.
Q_C	C’s selected-cluster range records and query key: $24P + 4 + Q_{\text{exact}}$ bytes on this 64-bit build.
Q_{exact}	Prefix minima and query sum: $8x + 8$ bytes when the optional bound is used at $x > 0$; zero otherwise.
Stored/query fields	Named field counts above exclude unused allocation capacity, overlapping result buffers and other process memory. Process peak RAM is measured separately.
G_q, g_q	Exact reference top- K IDs for query q , and how many of them are in the opened clusters.
$r_{B,q}$	Fraction of those opened reference IDs that B recovers. If $g_q = 0$, recall is zero.
$D_{q,\ell}$	Documents exposed across the opened clusters at C’s final prefix depth. The bars in $ G_q \cap D_{q,\ell} $ count reference IDs in that set.
n_q	Queries in the comparison. Average recall is total recovered reference IDs divided by $n_q K$, using the same full binary ranking and ID tie rule.

3 Experiments

The previous section describes the work each method performs. Here I vary the index and search settings, compare measured query time at the same recall requirement, and inspect the work counts to understand the results.

Each method can choose its own parameters. Within a comparison, the document codes, queries, score, reference IDs, recall requirement and RAM allowance stay fixed. The question is: at that same recall requirement, what is the fastest configuration we tested for each method?

3.1 Setup

I use two collections and several supported code lengths from their cached embeddings. Every method sees all 1,000 fixed queries in a pool. A different model or code length is a separate comparison:

Collection	Embedding model	Documents	Queries	Bits
MS MARCO	Nomic v1.5	1,000,000	1,000	64, 256, 768
Quora (BEIR)	Qwen3 0.6B	522,931	1,000	32, 256, 1024

I reused the cached Nomic document embeddings and selected 1,000 queries from MS MARCO’s official `dev.small` file [6, 4]. For Quora, I encoded the full BEIR corpus and selected 1,000 official test queries [7, 5]. Both random selections were fixed before search. Corpus duplicates remain; none of the selected Quora queries has identical text or ID in its corpus.

Each model produces a full vector once. I take supported prefixes, normalize them, and store their signs. Each length has its own reference: score every document and keep the best 100, resolving ties by document ID. The first one or ten IDs give the smaller result sets. Recall here measures recovery of these binary-score references; human relevance labels are a separate measure.

The machine is an Apple M5 Max with 128 GiB of memory. I use one native search query at a time and one Faiss search thread. The completed runs used a common 32 GB process cap. This is a recorded experiment setting, not the budget required by the formulas; we can evaluate other budgets such as 64 or 128 GB. The largest peak among the selected repeated runs was 505.6 MB, so none of those selections was close to the 32 GB cap. Reported query time starts with a cached query vector and ends with the returned IDs. It includes selecting clusters and searching their documents. Embedding generation and text reranking are outside this timer. Encoding and reference preparation finish before timed searches begin.

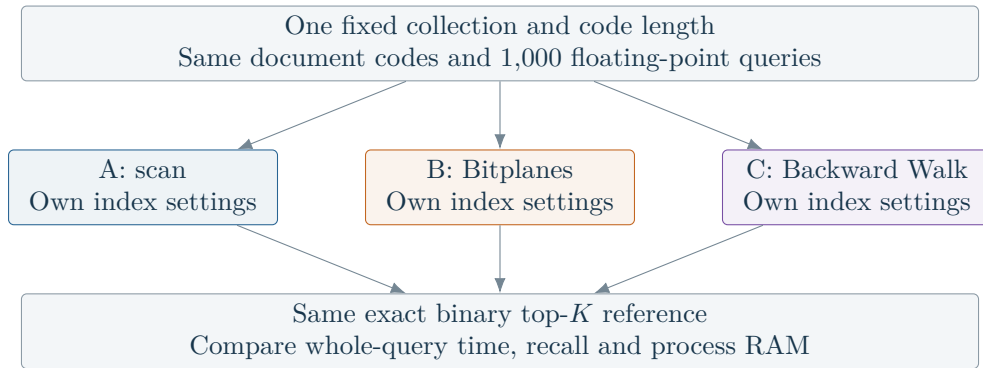


Figure 11: Only index and search settings differ within a pool. Changing the code length creates a new pool with its own reference ranking.

3.2 Parameter search

For A, I vary the cluster layout, cluster count C and probes P . The reference-cluster ranks give probe choices at the required recall targets. B receives those choices along with leaf sizes and node budgets. C receives them along with starting depths, candidate targets and the optional exact-stop rule.

Settings	Tested ranges and follow-ups
Routing	Float-trained clusters; binary-trained layouts also available to all methods in the short-code follow-ups. Cluster counts include 1, 16, 64, 256, 1024 and 4096.
Result count and recall	$K = 1, 10, 100$; targets 50%, 80%, 90%, 95%, 99%, with additional probes near high recall and whole-collection controls.
B	Initial leaves 128/2048 and budgets 512/8192; shorter codes also use leaves 8/32 and deeper-first ties. Probability 0/0.1/0.2 is a separate three-seed comparison.
C	Starting depths 0/1/4/16/32; candidate targets 1,000/10,000/100,000/half the collection/0. Exact-stop runs use target zero.

This is a summary of the settings tried, not a full cross-product at every width. The two 256-bit grids each have 570 settings; wider Qwen keeps $K = 1, 100$, and full-width Nomic focuses on $K = 100$ at 95% and 99% recall. Across the runs, 3,679 of 4,024 A/B/C settings qualify, plus all 72 probability settings. Three grids remain partial under their recorded time limits. The saved schedules list every attempted and missing setting.

I record complete-query time, recall, process RAM and the method’s work counts. For each target, I select the fastest qualifying setting in the main run, then repeat that exact choice in three fresh processes with varied run order. The tables report those repeated observations. Failed recall targets and incomplete runs stay in the saved data.

I also check direct alternatives. If B visits 512 clusters and scores all their rows without splitting, A must be allowed to use 512 probes too. C gets a depth-zero scan at A’s selected routing settings. These controls prevent a gap in the parameter grid from looking like an algorithmic advantage.

The equations identify useful quantities to inspect, such as the scored fraction and mask width. They do not turn the broad grid into a proven optimum, or supply the measured time constants in advance.

3.3 Which method is fastest?

The first table shows Qwen32 on all 522,931 Quora documents, returning one result. Each cell gives the repeated median query time and achieved recall. Bold marks the lowest median among methods that meet the row’s requirement.

Required recall	A (ms / recall)	B (ms / recall)	C (ms / recall)	Lowest median
80%	0.0308 / 80.3%	0.0295 / 81.9%	0.0318 / 80.3%	B
90%	0.0498 / 90.1%	0.0432 / 90.6%	0.0497 / 90.1%	B
95%	0.0805 / 95.0%	0.0520 / 95.3%	0.0799 / 95.0%	B
99%	0.1948 / 99.0%	0.0809 / 99.2%	0.1688 / 99.0%	B

B has the lowest repeated median at these four targets. The strongest difference is at 99%: 0.0809 ms for B versus 0.1948 ms for A, about 2.4 times faster. These recalls measure the 32-bit ranking. Even its exact top-1 answer agrees with the full-float best score on only 14% of queries; the code-length comparison below measures that separate quality tradeoff.

3.4 Parameters and work

At the 99% target, each method chooses a different routing layout. The selected configurations are:

Method	Clusters	Probes	Local setting	Recall	ms
A	4096	180	Scan opened rows	99.00%	0.1948
B	16	9	Leaf 128; budget 8192; deeper ties	99.20%	0.0809
C	1024	78	Start 4; target 0; exact stop	99.00%	0.1688

522,931 documents · 1,000 queries · 32 bits

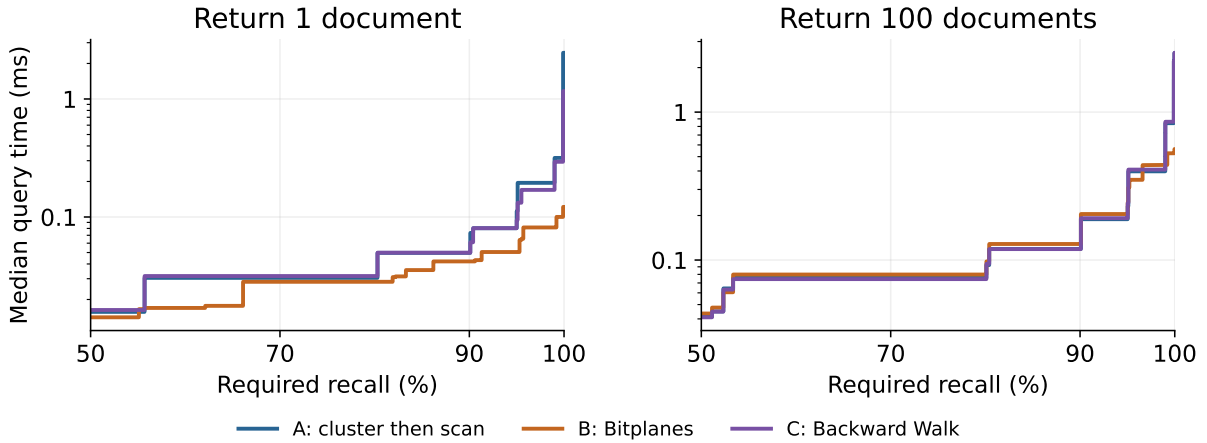


Figure 12: Qwen32 recall and query time, starting from cached vectors. The vertical axes use a log scale. The curves choose the fastest completed settings meeting each recall; the tables use their repeated timings.

Time is the median over the three repeated processes’ 3,000 query observations. B’s process medians range from 0.08025 to 0.08167 ms, below A’s 0.19223–0.19708 ms and C’s 0.16781–0.16994 ms. B’s finite budget gives measured 99.2% recall, not an exactness guarantee. C proves the answer within its opened clusters, but routing can still miss a global neighbor.

Now we can inspect the work behind those times. The entries below are mean counts per query; a dash means the operation is not used.

Method	Opened rows	Rows scored	Split-mask words	Leaf-mask words	Prefix depths	Boundary searches
A	24,181	24,181	—	—	—	—
B	295,031	818	53,693	5,838	—	—
C	41,147	26,153	—	—	3.511	494.988

B opens a much larger set than A, but scores only about 0.28% of those rows across the query batch. It visits about 116 candidate sets per query, including splits and leaves. The extra bitmap work therefore accompanies a large reduction in full scoring, and the complete query is faster even than A’s separately chosen layout.

C scores about 63.6% of its own opened rows. It is faster than A here despite scoring slightly more rows than A’s selected configuration. That comparison also changes routing: C has 1024 clusters and 78 probes, versus A’s 4096 and 180. We cannot attribute the whole time difference to prefix pruning alone. Searching inside fixed clusters and comparing complete queries with different routing settings answer different parts of the question.

The stored timers measure routing and complete index search, but do not separately time scoring, each mask pass and queue maintenance. I therefore report those operations as separate work counts. One mask word, one full score and one boundary search are different operations. The complete-query times show the net difference, but do not give an independently measured time for each operation.

3.5 Where does the result change?

If we ask for 100 results, the conclusion changes. The table below uses the same 99% recall requirement for every method and reports the repeated median of its selected setting.

Collection/model	Bits	A (ms)	B (ms)	C (ms)
MS MARCO/Nomic	64	1.915	2.071	1.875
MS MARCO/Nomic	256	13.453	13.866	14.645
MS MARCO/Nomic	768	45.088	45.217	45.196
Quora/Qwen	32	0.400	0.439	0.404
Quora/Qwen	256	3.564	3.721	4.039
Quora/Qwen	1024	22.863	23.098	22.930

Every entry reaches at least 99% recall again in its repeats. At these selected settings, B does no splitting and C starts at depth zero. Within each row, all three methods use the same 4096-cluster layout, probe count and number of document scores. C has a slightly lower median on Nomic64, but it avoids no scores there. Small timing differences between these scan-like paths do not establish a pruning advantage.

The earlier Qwen32 top-100 lead for B disappears when A also receives the binary-trained layouts. With the stronger routing, A opens 380 clusters and scores about 50,865 rows at 0.400 ms; B and C score the same rows at 0.439 and 0.404 ms. For one result, however, B’s pruning advantage remains, and C also beats the best tested A setting, although B is faster overall. This is why I compare each method with A’s own selected settings rather than only holding A at one convenient probe count.

Code length. The 32-bit result answers a different ranking question from the full 1024-value embedding. To measure that difference, I find the best full-float score over the entire corpus, then rescore each width’s exact binary top 100 using those full vectors. “Agrees” means its score is within 10^{-5} of the full-float best; equal-score documents need not have the same ID.

Model/collection	Bits	Binary top 1 agrees	Best score reached within binary top 100
Qwen/Quora	32	14.0%	73.0%
Qwen/Quora	256	75.8%	100.0%
Qwen/Quora	1024	88.6%	100.0%
Nomic/MS MARCO	64	17.6%	65.6%
Nomic/MS MARCO	256	48.0%	97.2%
Nomic/MS MARCO	768	70.2%	99.9%

For Qwen32, even the exact binary top-1 answer reaches the full-float best score on only 14% of queries. Keeping the binary top 100 raises that to 73%, compared with 100% at 256 bits. So a faster, high-recall search over 32-bit codes does not by itself preserve full-embedding results. These are score-agreement diagnostics, not human relevance scores or measured end-to-end quality for B or C. An approximate search may also miss some of those exact binary candidates.

Exact code matches. A separate global experiment removes routing entirely and requires the exact best binary result. I select one setting per method on all 1,000 Qwen32 queries, then split those same repeated observations by whether the query’s full 32-bit sign pattern exists in the corpus. There are 82 such queries and 918 without a matching code. The settings are not retuned for either group: B uses leaf 2048 with no node limit, and C starts at 32 with target zero and the exact-stop bound.

Query group	Queries	A (μ s)	B (μ s)	C (μ s)
All queries	1000	2481.667	124.084	1163.521
Complete code present	82	2481.250	128.521	4.875
Complete code absent	918	2481.708	123.750	1170.730

All three methods return the exact binary result in every group. On the 82 matching-code queries, C scores only 1.68 rows on average and stops after its first pair of boundary searches. This is a

real case where direct prefix lookup helps. A matching binary code does not imply identical text. But on the remaining 918 queries, it scores about 303,177 rows on average while widening the prefix. B therefore remains faster over the full 1,000-query batch. The $4.875 \mu\text{s}$ result belongs to that 8.2% subgroup under the same global settings; it is not C’s overall time or a comparison with separately tuned subgroup indexes.

3.6 Checking the calculations

I checked stored field sizes and operation counts separately from the timings. Across 13 saved work-check reports, all 890 index-storage checks match the formulas, including padding at cluster boundaries; the saved query-count checks report no failures. These checks do not assign a constant time to a mask word or document score.

The score gap helps explain why short codes behave differently. For each Qwen query, I divide the gap between the ideal binary score Q and the true best score by Q . The median gap is 2.60% at 32 bits, 14.48% at 256 bits and 16.15% at 1024 bits. When a retained score is close to Q , a smaller mismatch penalty can rule out a branch. The short-code result makes such pruning possible earlier; the representation table shows the separate cost of changing the ranking problem.

At the other extreme, one completed global Nomic768 setting with $K = 100$, leaf 2048 and budget 8192 scores all one million rows. It also visits 21.57 million split-mask words and 21.46 million final-mask words per query on average: about 43.03 million word visits without avoiding a full score. More available RAM would not remove that work.

Stopping depth. The exact-stop option uses the ideal score $Q = \sum_i |q_i|$ and the worst of the K retained scores, τ . At positive prefix depth ℓ , every unseen opened row disagrees with at least one retained sign. Its score is therefore at most

$$u_\ell = Q - 2 \min_{1 \leq i \leq \ell} |q_i|.$$

C can stop when $\tau > u_\ell$, with the code’s rounding margin and tie rule. This proves the result within the opened clusters; it does not correct routing misses.

For the global Qwen32 exact-stop runs, let y_q be the deepest stored prefix whose unseen bound is below the saved exact best score, or zero if none qualifies. A walk starting at x then ends at $\ell_q = \min(x, y_q)$. Counting the corresponding sorted prefix range gives the number of scored rows; the visited-depth and lookup counts follow from $x - \ell_q + 1$. All four predictions match all 3,000 query records: the same 1,000 queries at starts 1, 4 and 32.

Start x	Mean final depth	Mean rows scored	Mean visited depths	Mean boundary searches
1	0.665	353,270.1	1.335	2.000
4	1.497	290,211.1	3.503	6.336
32	4.505	278,316.6	28.495	56.320

For $K = 1$, 665 queries prove a stop above the root. Starting at 32 reduces the mean number of scores, but requires many more boundary searches. The prediction uses reference scores only in this offline check; actual search uses the scores it has found. The saved $K = 100$ predictions were not measured in this global run, so they are not included in the 3,000 matching records.

Memory at the selected settings. For the Qwen32 top-1 settings, the verified native fields plus router storage and the largest repeated-process memory peak are:

Method	Native and router fields (MB)	Repeated-process peak (MB)
A	8.924	77.971
B	10.463	72.139
C	10.598	77.529

MB here means one million bytes. The field count includes codes, IDs, bitplanes or prefix keys, and router centroids and labels. It excludes unused capacity and other process memory, so it must not be confused with the measured peak. Even the selected Nomic768 B setting uses 225.1 MB of stored fields and a 505.6 MB process peak. The 505.6 MB peak is the largest across all selected repeated settings. Every one also fits a 64 GB or 128 GB allowance, so raising the feasibility limit alone leaves these recorded comparisons unchanged. This does not predict the performance of a larger, untested index.

Random visits. I also held the Qwen32 layout at 64 float-trained clusters, opened 16, and kept leaf size 32 while varying B’s exploration probability. The following top-1 results use three seeds per probability. Recall is their mean and time is the median of their query-time medians.

Probability	Budget 128		Budget 8192	
	Recall	ms	Recall	ms
0	87.30%	0.0460	95.30%	0.0766
0.1	83.33%	0.0479	95.30%	0.0857
0.2	70.90%	0.0490	95.30%	0.0959

With the smaller budget, random visits change which branches are processed before the limit, and recall falls. With the larger budget, recall stays at 95.3% while time increases. The Nomic64 runs show no consistent time gain at matching recall either. This experiment does not establish that randomness can never help; it does not improve the tested tradeoff here.

3.7 Code and reproduction

The C++ scorer is shared by all three methods. A and B are in `cpp/index.cpp`; C is in `cpp/prefix_index_v2.cpp`. The Python worker in `experiments/prefix_batch_worker_v2.py` times routing and search, records the work counts, and compares returned IDs with the exact references.

After preparing the arrays using `experiments/prepare_prefix_pools_v2.py`, we can run one complete setup with this Python flow:

```
import json
from pathlib import Path
from experiments import prefix_study_v2 as study
path = Path("configs/prefix-study-v2/qwen-quora-d256.json")
config = json.loads(path.read_text())
output = Path("results/my-quora-run")
study.prepare(config, output) # Build clusters and the schedule.
study.run(output)            # Measure the declared settings.
study.summarize(output)       # Select by recall and time.
study.repeat(output)          # Repeat those choices three times.
```

Full commands, input hashes, selected settings and raw query records are linked from `results/prefix-study-2026-09-13/README.md`. The data and encoding recipe is in `docs/expanded-pool-preparation.md`. The root `README.md` maps the paper to its code, including the exact-stop prediction and full-float agreement checks.

4 Limitations and next steps

The timings describe this CPU implementation. All three methods reuse the same C++ lookup-table scorer, which adds entries using float64. Exa’s production system and its SIMD kernels are not reproduced here. A faster scoring kernel can change the balance between scoring documents and maintaining an index search.

I hold the collection, model and code length fixed when choosing each method’s parameters. Shorter codes change the reference ranking, so their binary recall does not establish the same retrieval quality as a longer representation. The source datasets provide human relevance labels for that separate evaluation.

These are static indexes built before querying. The experiment does not measure document updates or concurrent requests. In particular, keeping rows sorted still costs work when new documents are added. The memory formulas count the binary search index and its query buffers; an application that keeps uncompressed vectors or document text in RAM must add that storage.

Prefix expansion also appears in LSH Forest [8] and PUFFINN [9]. Those methods use multiple hash indexes and different stopping rules. Their recall guarantees do not transfer to a single prefix in the embedding’s original coordinate order.

5 Conclusion

For Qwen32 with one returned result, B takes 0.081 ms at 99.2% binary recall, versus A’s 0.195 ms and C’s 0.169 ms at 99.0%. All methods can choose binary-trained clusters. With 100 results at the 99% target, the selected B and C settings instead become scans. C has a narrower exact-search benefit: it takes 4.875 μ s on the 82 queries whose complete code exists, while B remains faster over all 1,000 queries.

These results identify useful tested settings, not a universal optimum. Qwen32’s exact binary top 1 agrees with the full-float best score on only 14% of queries, so the short-code speed does not preserve full-embedding quality. For a fixed representation and recall requirement, the decision remains whether the scores avoided pay for the added mask or prefix work. The measured counts let us check that explanation directly.

References

- [1] The Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS, 2022. <https://arxiv.org/abs/2205.13147>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model card, accessed 13 September 2026. The saved manifest identifies the exact model revision. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. <https://arxiv.org/abs/2506.05176>. Encoding recipe: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [6] P. Bajaj et al. *MS MARCO: A Human Generated MACHINE Reading COMprehension Dataset*. 2016; revised 2018. <https://arxiv.org/abs/1611.09268>. Passage-ranking data: <https://github.com/microsoft/MSMARCO-Passage-Ranking>.

-
- [7] N. Thakur et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2104.08663>. Dataset catalog: <https://github.com/beir-cellar/beir>.
 - [8] M. Bawa, T. Condie and P. Ganesan. *LSH Forest: Self-Tuning Indexes for Similarity Search*. WWW, 2005, pp. 651–660. <https://www.cs.princeton.edu/courses/archive/spring06/cos592/bib/LSHForest-bawa05.pdf>.
 - [9] M. Aumüller, T. Christiani, R. Pagh and M. Vesterli. *PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors*. ESA, 2019. <https://doi.org/10.4230/LIPIcs.ESA.2019.10>.